



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Linear and Logistic Regression

EIN087B - Data Science

Profesor: Jorge Portilla G.
jorge.portilla@usm.cl

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Regresión Lineal

- ▶ Es uno de los métodos que aplican estadística más utilizados.
- ▶ Se usa para describir la relación entre el **promedio (tendencia)** de una variable y los valores tomados por otras variables $\Rightarrow$ **regresión**.
- ▶ A veces tenemos varios modelos plausibles, y queremos seleccionar cuál es el más apropiado $\Rightarrow$ **Análisis de varianza**.

Podríamos aproximar el tiempo total de viaje diario y como una función lineal de la distancia recorrida y el número de entregas realizadas:

$$f(x) = \beta_0 + \beta_1 x^{(1)} + \beta_2 x^{(2)},$$

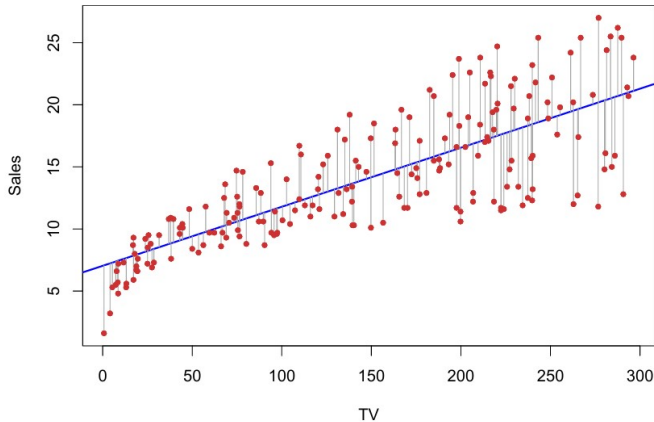
donde β_i , $i = 0, 1, 2$ son los parámetros del modelo lineal.

- ▶ Así, el espacio de funciones lineales que mapean de $\mathcal{X}$ a $\mathcal{Y}$ es **paramétrico**.
- ▶ Definimos $x^{(0)} = 1$ para definir el modelo lineal en forma matricial:

$$f(x) = \sum_{i=0}^I \beta_i x^{(i)} = \beta^T x,$$

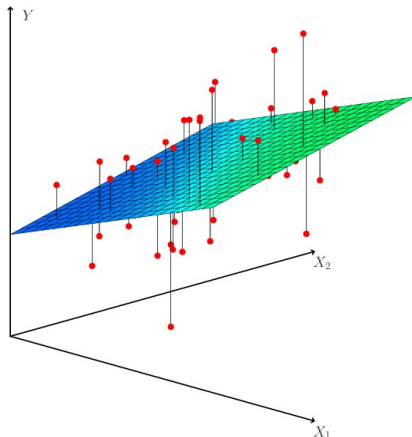
donde I es el número de características.

Ejemplo gráfico de regresión lineal



- ▶ Encontrar un hiperplano.
- ▶ Criterio mínimos cuadrados o *least square*.

Ejemplo gráfico de regresión lineal



En un entorno tridimensional, con dos predictores y una respuesta, la línea de regresión por mínimos cuadrados se convierte en un plano. El plano se elige para minimizar la suma de las distancias verticales al cuadrado entre cada observación.

¿Cómo seleccionamos los β 's?

- Podemos obtener los parámetros del modelo minimizando la función de pérdida cuadrática sobre el conjunto de entrenamiento:

$$J(\beta) = \frac{1}{2} \sum_{m=1}^M (f(x_m) - y_m)^2$$

donde $\beta = (\beta_0, \beta_1, \dots, \beta_I)^T$.

- Necesitamos elegir β que minimice $J(\beta)$.

Algoritmo Least Mean Squares (LMS)

- Este problema puede expresarse en forma matricial:

$$J(\beta) = \frac{1}{2}(Y - X\beta)^T(Y - X\beta),$$

donde X es una matriz de $N \times (I + 1)$.

Desde cálculo matricial

- Si x es un vector columna:

$$\frac{\partial u^T v}{\partial x} = \frac{\partial u}{\partial x} \cdot v + \frac{\partial v}{\partial x} \cdot u$$

- $\frac{\partial Ax}{\partial x} = A^T$

- Entonces,

$$\frac{\partial J(\beta)}{\partial \beta} = -X^T(Y - X\beta)$$
$$\frac{\partial^2 J(\beta)}{\partial \beta \partial \beta^T} = X^T X$$

- Igualando la primera derivada a cero obtenemos las ecuaciones normales:

$$X^T(Y - X\beta) = 0 \implies \hat{\beta} = (X^T X)^{-1} X^T Y$$

Esta es la solución analítica.

- Entonces,

$$\frac{\partial J(\beta)}{\partial \beta} = -X^T(Y - X\beta)$$
$$\frac{\partial^2 J(\beta)}{\partial \beta \partial \beta^T} = X^T X$$

- Igualando la primera derivada a cero obtenemos las ecuaciones normales:

$$X^T(Y - X\beta) = 0 \implies \hat{\beta} = (X^T X)^{-1} X^T Y$$

Esta es la solución analítica.

- ¿Qué sucedería si la solución analítica es costosa de calcular debido a los grandes volúmenes de datos?

- Entonces,

$$\frac{\partial J(\beta)}{\partial \beta} = -X^T(Y - X\beta)$$
$$\frac{\partial^2 J(\beta)}{\partial \beta \partial \beta^T} = X^T X$$

- Igualando la primera derivada a cero obtenemos las ecuaciones normales:

$$X^T(Y - X\beta) = 0 \implies \hat{\beta} = (X^T X)^{-1} X^T Y$$

Esta es la solución analítica.

- ¿Qué sucedería si la solución analítica es costosa de calcular debido a los grandes volúmenes de datos?
- Métodos iterativos como el descenso del gradiente se convierten en una opción más viable y eficiente.

- Asumiendo que el modelo lineal es correcto, es decir, $Y = X\beta + \epsilon$ para algún β desconocido.
- Además, asumimos que $E[\epsilon] = 0$ y $\text{Cov}(\epsilon) = E[\epsilon\epsilon^T] = \sigma^2 I$ (ruido no correlacionado). De la solución de mínimos cuadrados $\beta_{ls} = (X^T X)^{-1} X^T Y$. Así,

$$\begin{aligned} E[\beta_{ls}] &= E \left[(X^T X)^{-1} X^T Y \right] \\ &= (X^T X)^{-1} X^T E[Y] = (X^T X)^{-1} X^T X \beta = \beta, \end{aligned} \tag{5}$$

$$\begin{aligned} E[\beta_{ls}] &= E \left[(X^T X)^{-1} X^T Y \right] \\ &= (X^T X)^{-1} X^T E[Y] = (X^T X)^{-1} X^T X \beta = \beta, \end{aligned} \quad (5)$$

- Es decir, β_{ls} es un estimador insesgado de β .
- Además,

$$\begin{aligned} \beta_{ls} - E[\beta_{ls}] &= (X^T X)^{-1} X^T Y - \beta \\ &= (X^T X)^{-1} X^T Y - (X^T X)^{-1} X^T X \beta \\ &= (X^T X)^{-1} X^T (Y - X \beta) \\ &= (X^T X)^{-1} X^T \epsilon \end{aligned} \quad (6)$$

- Así, a partir de la suposición $E[\epsilon\epsilon^T] = \sigma^2 I$ obtenemos

$$\begin{aligned}\text{Cov}(\beta_{ls}) &= E \left[(\beta_{ls} - E[\beta_{ls}]) (\hat{\beta}^{ls} - E[\beta_{ls}])^T \right] \\ &= E \left[(X^T X)^{-1} X^T \epsilon \epsilon^T X (X^T X)^{-1} \right] \\ &= (X^T X)^{-1} X^T E \left[\epsilon \epsilon^T \right] X (X^T X)^{-1} \\ &= \sigma^2 (X^T X)^{-1} X^T X (X^T X)^{-1} \\ &= \sigma^2 (X^T X)^{-1}\end{aligned}$$

- Típicamente, la varianza σ^2 se estima como

$$\hat{\sigma}^2 = \frac{1}{N - I - 1} \sum_{m=1}^M (y_m - \hat{y}_m)^2$$

Podríamos aproximar el tiempo total de viaje diario y como una función lineal de la distancia recorrida y el número de entregas realizadas:

$$f(x) = \beta_0 + \beta_1 x^{(1)} + \beta_2 x^{(2)},$$

donde β_i , $i = 0, 1, 2$ son los parámetros del modelo lineal.

- ▶ Así, el espacio de funciones lineales que mapean de $\mathcal{X}$ a $\mathcal{Y}$ es **paramétrico**.
- ▶ Definimos $x^{(0)} = 1$ para definir el modelo lineal en forma matricial:

$$f(x) = \sum_{i=0}^I \beta_i x^{(i)} = \beta^T x,$$

donde I es el número de características.

- ▶ En aplicaciones reales de aprendizaje automático, tenemos que trabajar con espacios de entrada de alta dimensionalidad que comprenden muchas variables de entrada.
- ▶ Por lo tanto, con un número exponencialmente grande de celdas, necesitaríamos una cantidad exponencialmente grande de ejemplos de entrenamiento para asegurarnos de que las celdas no estén vacías.
- ▶ En problemas de regresión, nos gustaría modelar el problema considerando las dependencias entre las características de entrada:

$$f(\mathbf{x}, \boldsymbol{\beta}) = \beta_0 + \sum_{i=1}^I \beta_i x^{(i)} + \sum_{i=1}^I \sum_{j=1}^I \beta_{ij} x^{(i)} x^{(j)} + \sum_{i=1}^I \sum_{j=1}^I \sum_{k=1}^I \beta_{ijk} x^{(i)} x^{(j)} x^{(k)}$$

El descenso del gradiente es un método iterativo que puede ser utilizado cuando la solución analítica es costosa de calcular o cuando se trabaja con grandes volúmenes de datos.

► Solución Analítica (Mínimos Cuadrados)

- Proporciona una solución exacta: $\beta = (X^T X)^{-1} X^T Y$.
- Costoso computacionalmente para grandes conjuntos de datos.

► Descenso del Gradiente Estocástico (SGD)

- Actualiza los parámetros iterativamente utilizando muestras individuales o pequeños lotes.
- Actualización para una muestra $(x^{(i)}, y^{(i)})$:

$$\beta := \beta + \alpha x^{(i)} (y^{(i)} - x^{(i)T} \beta)$$

- Eficiente para grandes volúmenes de datos.
- No requiere invertir matrices grandes.

Algoritmo de Descenso de Gradiente

El descenso del gradiente es un método iterativo que puede ser utilizado cuando la solución analítica es costosa de calcular o cuando se trabaja con grandes volúmenes de datos.

- Consideremos el algoritmo de descenso de gradiente que comienza con un β inicial y realiza repetidamente:

$$\beta_i^{p+1} = \beta_i^p - \alpha \frac{\partial}{\partial \beta_i} J(\beta)$$

- Calculemos la derivada de la función de pérdida para un solo ejemplo de entrenamiento (x_m, y_m) :

$$\begin{aligned} \frac{\partial}{\partial \beta_i} J(\beta) &= \frac{\partial}{\partial \beta_i} \frac{1}{2} (f(x_m) - y_m)^2 \\ &= (f(x_m) - y_m) \cdot \frac{\partial}{\partial \beta_i} \left(\sum_{i=0}^I \beta_i x^{(i)} \right) \\ &= (f(x_m) - y_m) x^{(i)} \end{aligned}$$

Explicación - Derivada de la Función de Pérdida

- Considere la función de pérdida para un solo ejemplo de entrenamiento (x_m, y_m) :

$$J(\beta) = \frac{1}{2}(f(x_m) - y_m)^2$$

- Queremos encontrar la derivada de $J(\beta)$ con respecto a β_i :

$$\frac{\partial J(\beta)}{\partial \beta_i} = \frac{\partial}{\partial \beta_i} \left(\frac{1}{2}(f(x_m) - y_m)^2 \right)$$

Explicación - Derivada de la Función de Pérdida

- Considere la función de pérdida para un solo ejemplo de entrenamiento (x_m, y_m) :

$$J(\beta) = \frac{1}{2}(f(x_m) - y_m)^2$$

- Queremos encontrar la derivada de $J(\beta)$ con respecto a β_i :

$$\frac{\partial J(\beta)}{\partial \beta_i} = \frac{\partial}{\partial \beta_i} \left(\frac{1}{2}(f(x_m) - y_m)^2 \right)$$

- Aplicamos la regla de la cadena:

$$\frac{\partial J(\beta)}{\partial \beta_i} = (f(x_m) - y_m) \cdot \frac{\partial (f(x_m) - y_m)}{\partial \beta_i}$$

- Dado que y_m es constante y $f(x_m) = \sum_{j=0}^I \beta_j x_m^{(j)}$, tenemos:

$$\frac{\partial (f(x_m))}{\partial \beta_i} = x_m^{(i)}$$

- Por lo tanto, la derivada es:

$$\frac{\partial J(\beta)}{\partial \beta_i} = (f(x_m) - y_m) \cdot x_m^{(i)}$$

Algoritmo de Descenso de Gradiente (2)

- ▶ Entonces, para un solo ejemplo:

$$\beta_i^{p+1} = \beta_i^p - \alpha(f(x_m) - y_m)x_m^{(i)}$$

- ▶ Esta regla se llama **regla de aprendizaje de Widrow-Hoff**.
- ▶ Observe que la cantidad de la actualización es proporcional al error:
 $(f(x_m) - y_m)$

$\beta_i^{(p+1)}$ se refiere al i -ésimo parámetro en la iteración $p + 1$.

Algoritmo de Descenso de Gradiente por Lotes

- Podemos aplicar la regla de aprendizaje anterior para el [train set](#):

Algoritmo: Batch gradient descent algorithm

1. **repeat**

2.

$$\beta_i^{(p+1)} = \beta_i^p - \alpha \sum_{m=1}^M (f(x_m) - y_m) x_m^{(i)} \quad (\text{para cada } i)$$

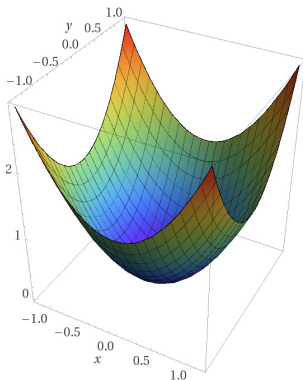
3. **until** Convergence

- En general, el descenso de gradiente puede alcanzar un mínimo local.
- Sin embargo, J^1 es una función convexa. Por lo tanto, el problema de optimización tiene solo un óptimo global.

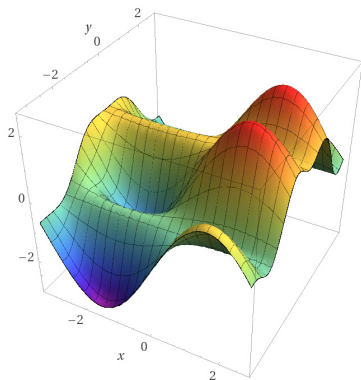
$\beta_i^{(p+1)}$ se refiere al i -ésimo parámetro en la iteración $p + 1$.

¹El error cuadrático medio es una función cuadrática en términos de los parámetros β . Una función es convexa si la segunda derivada de la función es siempre no negativa.

Conexas y ninguna

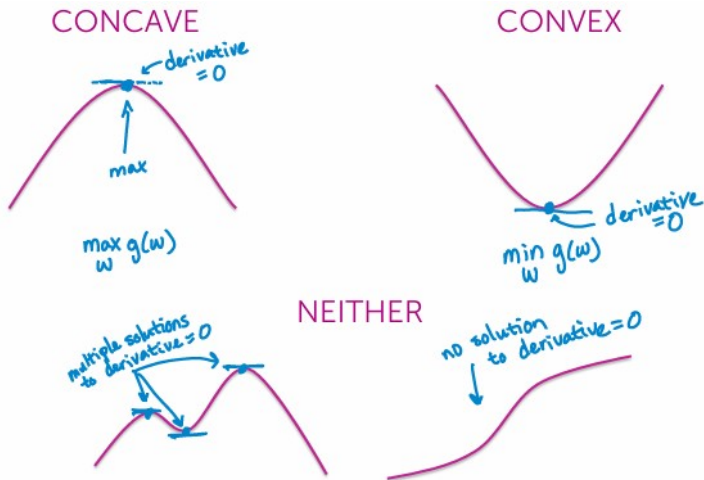


Computed by WolframAlpha



Computed by WolframAlpha

Funciones convexas y cóncavas



3

³Fox, E., Guetrin, C. Simple regression: Linear regression with one input. Machine Learning Specialization, University of Washington.

Algoritmo de Descenso de Gradiente Estocástico

- Podemos modificar la regla de aprendizaje anterior para cada ejemplo:

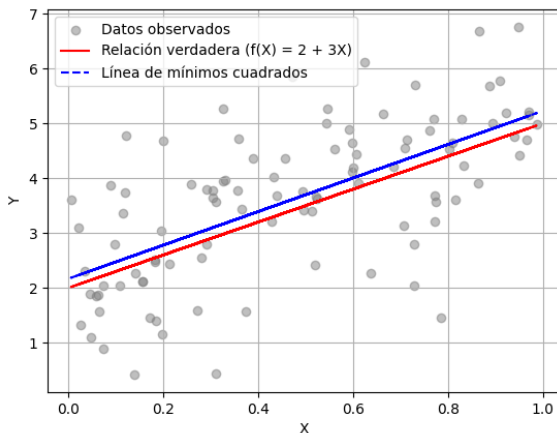
Algoritmo: Descenso de Gradiente Estocástico

1. **repeat:**
2. **for** $m := 1$ to M **do:**
 - $\beta_i^{(p+1)} := \beta_i^p - \alpha(f(x_m) - y_m)x_m^{(i)}$ (para cada i)
 - $\beta_i^p := \beta_i^{(p+1)}$
3. **until** Convergence

- Este método se llama descenso de **gradiente estocástico** o **online**.
- Usualmente, esta técnica converge más rápido que el **batch gradient descent algorithm**.
- Sin embargo, usando un valor fijo para α puede que nunca converja al mínimo de $J(\beta)$, oscilando alrededor de él.
- Para evitar este comportamiento, se recomienda disminuir lentamente α a cero durante las iteraciones.
- **Nota:** m representa el índice de una muestra específica en el conjunto de datos (M es el número total de ejemplos).

- ▶ Generamos datos siguiendo el modelo $Y = 2 + 3X + \epsilon$, donde ϵ es un término de error aleatorio con media cero.
- ▶ **Generación de Datos:**
 - Creamos 100 valores aleatorios de X y calculamos los valores correspondientes de Y usando el modelo mencionado.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4
5 np.random.seed(42)
6 X = np.random.rand(100, 1)
7
8 epsilon = np.random.randn(100, 1)
9 # Error aleatorio normal con media 0 y STD 1
10 Y = 2 + 3 * X + epsilon
11
12 reg = LinearRegression()
13 reg.fit(X, Y) # Ajustar el modelo de mínimos cuadrados
14 Y_pred = reg.predict(X)
```



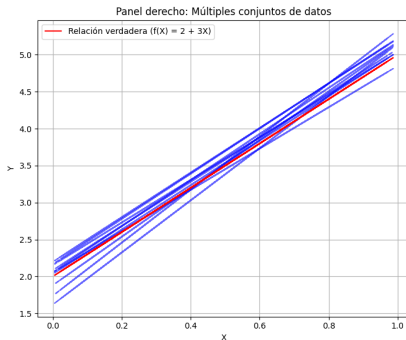
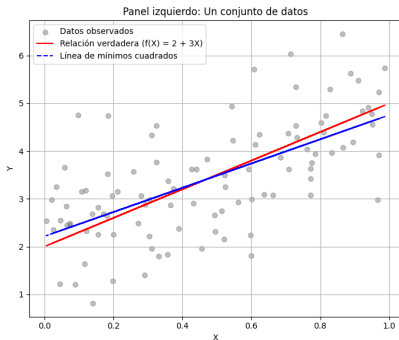
- **Relación Verdadera vs. estimación:** La línea roja representa la verdadera relación teórica $f(X) = 2 + 3X$, mientras que la línea azul discontinua es la estimación basada en los datos observados.

- **Variabilidad de las estimaciones:** Diferentes conjuntos de datos del mismo modelo generan diferentes líneas de mínimos cuadrados.
- **Aplicaciones reales:** La verdadera relación $f(X)$ no se conoce en aplicaciones reales. Variabilidad y la incertidumbre en las estimaciones.

```
1  for _ in range(10):  
2      epsilon = np.random.randn(100, 1)  # Generar nuevo error aleatorio  
3      Y = 2 + 3 * X + epsilon  
4      reg.fit(X, Y)  
5      Y_pred = reg.predict(X)  
6      plt.plot(X, Y_pred, color='blue', alpha=0.6, linestyle='--')
```

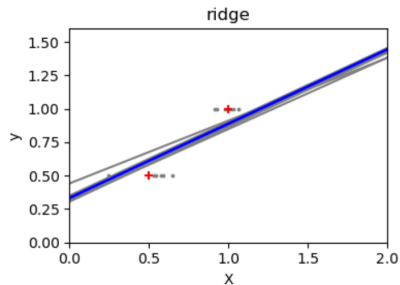
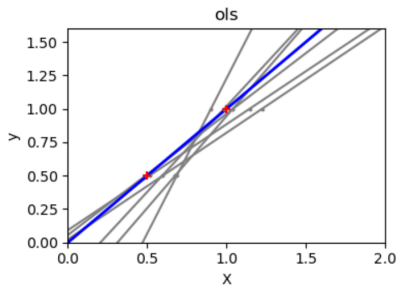
Ejemplo: Regresión Lineal

- **Variabilidad de las estimaciones:** Diferentes conjuntos de datos del mismo modelo generan diferentes líneas de mínimos cuadrados.
- **Aplicaciones reales:** La verdadera relación $f(X)$ no se conoce en aplicaciones reales. Variabilidad y la incertidumbre en las estimaciones.



- ▶ Debido a los pocos puntos de cada dimensión y a la línea recta que utiliza la regresión lineal para seguir estos puntos lo mejor posible, el ruido en las observaciones provocará una gran varianza.
- ▶ La pendiente de cada línea puede variar bastante para cada predicción debido al ruido inducido en las observaciones.
- ▶ Estudiaremos la **regresión Ridge** que consiste en una penalización a los coeficientes de regresión lineal.
- ▶ La penalización reduce (**penalising shrinks**) el valor de los coeficientes de regresión. La pendiente de la predicción es mucho más estable y la varianza de la propia línea se reduce considerablemente, en comparación con la de la regresión lineal estándar.

Regresión Ridge vs Regresión Lineal



4

⁴https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols_ridge_variance.html#sphx-glr-auto-examples-linear-model-plot-ols-ridge-variance-py

Parámetros de regularización en regresión lineal Ridge - Lasso.

- ▶ ¿Qué es la multicolinealidad?
 - Ocurre cuando dos o más variables independientes en un modelo de regresión están altamente correlacionadas, lo que significa que una variable puede ser predicha linealmente a partir de las otras con un alto grado de precisión.
- ▶ Problemas causados por la multicolinealidad:
 - Puede inflar las varianzas de los coeficientes estimados, lo que hace que los coeficientes sean inestables y difíciles de interpretar.
 - Puede llevar a que el modelo no pueda estimar los efectos de las variables independientes correctamente.
- ▶ Detección de la multicolinealidad:
 - Se puede detectar usando el **número de condición** de la matriz de diseño o mediante el **factor de inflación de la varianza (VIF)**⁵.
- ▶ Relación con la regresión Ridge:
 - La regresión Ridge se utiliza para corregir los problemas de multicolinealidad al penalizar la magnitud de los coeficientes, lo que reduce su varianza.

⁵Una forma más simple, el rango de una matriz.

- ▶ También conocida como **regularización de Tikhonov**.
- ▶ El **rango de una matriz** nos dice si las columnas de una matriz son exactamente linealmente independientes (una columna copia de otra columna, entre otras).
 - El rango es estricto, si una columna esta muy cercana a ser igual a otra columna, la marca como diferente. Esto presenta problemas en el least squares.
- ▶ El rango avisa que tengo un problema, y estaremos tratando de invertir una matriz singular⁶, entonces el problema de least squares es **ill-posed**.
- ▶ Tener un problema ill-posed significa que ante pequeños cambios o ruido numérico en los datos de entrada nos conduce a distintas soluciones.

⁶Matriz cuadrada que no tiene inversa. Muchas posibles soluciones.

Regresión Ridge (2)

- ▶ ¿Como saber si debo usar una regresión ridge?
 - Número de condición de una matriz.
- ▶ El **número de condición** de una matriz A , denotado como $\kappa(A)$.
- ▶ Indica qué tan cerca está la matriz de ser singular.
- ▶ Un valor alto de $\kappa(A)$ sugiere que el sistema puede ser ill-posed (significa que pequeñas perturbaciones en los datos pueden causar grandes cambios en la solución).
- ▶ El número de condición se calcula generalmente usando la norma⁷ L_2 ⁸ de la matriz, de la siguiente manera:

$$\kappa(A) = \|A\|_2 \cdot \|A^{-1}\|_2$$

Donde:

- $\|A\|_2$ es la norma L_2 de la matriz A .
- $\|A^{-1}\|_2$ es la norma L_2 de la matriz inversa A^{-1} .

⁷Número no negativo que describe la extensión de este vector en el espacio.

⁸La norma L_2 calcula la distancia de un vector desde el origen en un espacio vectorial. Se conoce como la norma euclidiana, ya que se calcula como la distancia euclidiana desde el origen. Se calcula como raíz cuadrada de la suma de los cuadrados de los elementos del vector.

Ejemplo problema ill-posed

Emisión de banda ancha (BEMI)

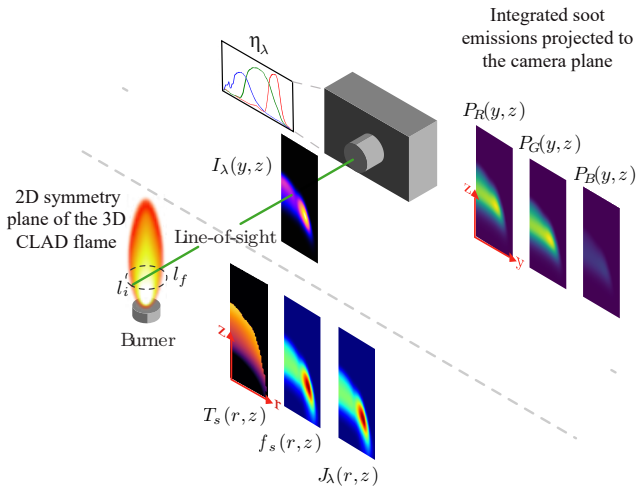
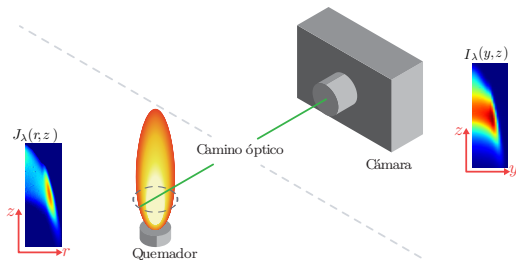


Figure 1: Schematic of BEMI setup.

Emisión de banda ancha (BEMI)



La emisión local del hollín a una longitud de onda dada λ se representa como:

$$J_{\lambda}(T_s, f_s) = \kappa_{\lambda}(f_s) I_{\lambda}^{bb}(T_s) \quad (1)$$

- κ_{λ} corresponde al coeficiente de absorción de hollín representado como^a
 $\kappa_{\lambda} = f_s 6\pi E_{(m)} / \lambda$, con $E_{(m)}$ siendo la función de absorción de hollín.
- I_{λ}^{bb} es la intensidad de radiación de cuerpo negro.

La intensidad de la radiación en la línea de visión detectada por la cámara representada por:

$$I_{\lambda}(y, z) = \int_l J_{\lambda}(l, z) \exp\left(-\int_{l'} \kappa_{\lambda}(l', z) dl'\right) dl, \quad (2)$$

donde y, z son las coordenadas de píxel de la cámara, y l' es el camino óptico desde la posición l hasta los sensores.

^aM. Modest, Radiative heat transfer. Academic press, 2013.

Ejemplo de problema ill-posed - Emisión de banda ancha (BEMI)

Cuando se utilizan sensores RGB, se integra la expresión correspondiente a I_λ (despreciando la auto-absorción) sobre las longitudes de onda de detección de la cámara RGB.

$$P_i(y, z) = \int_{\lambda_i} \eta_{i,\lambda} I_\lambda(y, z) d\lambda = \int_l \int_{\lambda_i} \eta_{i,\lambda} J_\lambda(l, z) d\lambda dl = \int_l H_i(l, z) dl, \quad (3)$$

donde $i \in \{R, G, B\}$ representan los canales Red, Green, Blue.

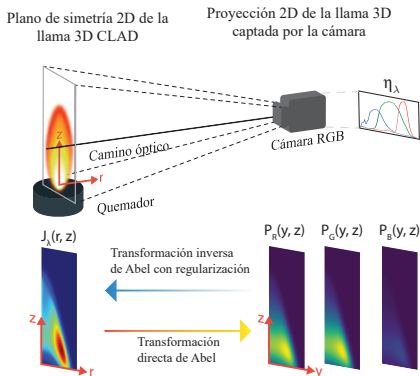
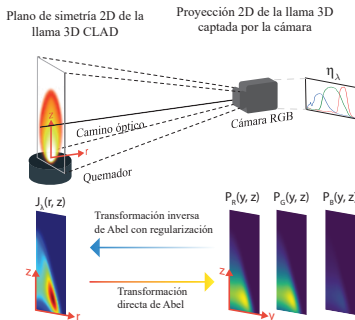


Figura: Esquema de las mediciones integradas de llama en la línea de visión del fotodetector.

Ejemplo de problema ill-posed - Emisión de banda ancha (BEMI)

Dada la geometría axisimétrica de la llama y utilizando un cambio de variable en la integral, se convierte en:)

$$P_i(y) = 2 \int_y^R \frac{H_i(r)r}{\sqrt{r^2 - y^2}} dr, \quad (4)$$



- ▶ En forma matricial como $\mathbf{P}_i = \mathbf{A}\mathbf{H}_i$, donde $\mathbf{H}_i$ corresponde a los parámetros desconocidos, y $\mathbf{A}$ corresponde a una matriz con los parámetros geométricos de la medición.
- ▶ Es un tipo de ecuación integral de Abel, y su solución implica resolver un problema inverso con un sistema de ecuaciones ill-posed.
- ▶ En lugar de realizar la transformada inversa utilizando directamente la inversa de la matriz $\mathbf{H}_i = \mathbf{A}^{-1}\mathbf{P}_i$, se utiliza el método de onion-peeling (OP) con un paso adicional de regularización de Tikhonov $((\mathbf{A}^T \mathbf{A} + \alpha \mathbf{L}^T \mathbf{L})\tilde{\mathbf{H}} = \mathbf{A}^T \mathbf{P})$.

Regresando a la clase...

Regresión Ridge (2)

- Reduce los valores de los coeficientes introduciendo un término de regularización,

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \left\{ \sum_{m=1}^M \left(y_m - \beta_0 - \sum_{i=1}^I x_{mi} \beta_i \right)^2 + \lambda \sum_{i=1}^I \beta_i^2 \right\} \quad (9)$$

- $\lambda \geq 0$ controla el nivel de regularización⁹.
- El término $\lambda \sum_{i=1}^I \beta_i^2$ suma una penalización a la función de costo si los coeficientes son grandes.
 - Cuando el algoritmo minimiza $\hat{\beta}^{\text{ridge}}$, no solo intenta ajustar los datos $\hat{y}_m - y_m$, sino que también trata de mantener los coeficientes lo más pequeños posible β_i .
- Es dependiente de la escala de los datos. Por lo tanto, la normalización de los datos es estrictamente necesaria.
- β_0 no está acotado (se aplica únicamente a los coeficientes de las variables predictoras, no al término de intercepción).

⁹ Penaliza la magnitud de los coeficientes β_i

- ▶ Podemos transformar los datos usando $x_{mi} \leftarrow (x_{mi} - \bar{x}_i)$
- ▶ Así, estimamos $\beta_0 = \bar{y} = \frac{1}{M} \sum_{m=1}^M y_m$ (valor esperado de y cuando $x = 0$)
- ▶ Ahora X tiene I columnas. Entonces, la ecuación de pérdida cuadrática en forma matricial puede expresarse como

$$RSS(\lambda) = \frac{1}{2} \left((Y - X\beta)^T (Y - X\beta) + \lambda \beta^T \beta \right)$$

- ▶ La solución de esta ecuación es

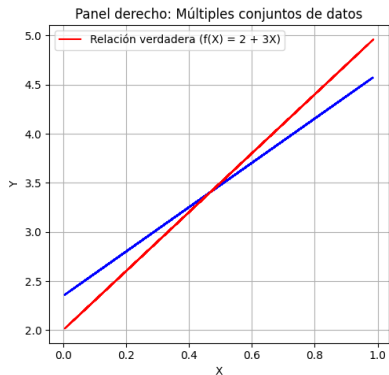
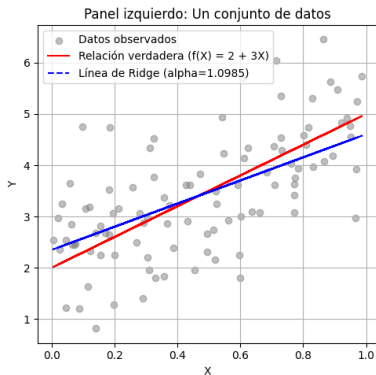
$$\hat{\beta}^{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T Y$$

- ▶ Ahora el problema es no singular¹⁰.
- ▶ Para entradas ortonormales tenemos, $\hat{\beta}^{\text{ridge}} = \gamma \hat{\beta}^{LS}$, $0 \leq \gamma \leq 1$ ¹¹

¹⁰Se indica que la inclusión del término λI en la matriz $X^T X$ resuelve cualquier problema de singularidad, asegurando que la matriz es invertible. Esto significa que siempre podemos encontrar una solución única para los coeficientes β .

¹¹Cuando las variables independientes son ortonormales, la solución de Ridge es simplemente un escalar γ multiplicado por la solución de los mínimos cuadrados ordinarios (β^{LS}). Este escalar γ depende de λ

Ejemplo anterior con Ridge



- ▶ Least absolute shrinkage and selection operator.
- ▶ Solución *sparse*, encontramos los mejores descriptores para resolver el problema. Modelo interpretable.
- ▶ Coeficientes con valor cero¹².

¹²Se eliminan características irrelevantes, puede ser una buena opción porque lleva algunos coeficientes exactamente a cero.

- Se define el estimador **lasso** como:

$$\hat{\beta}_{\text{lasso}} = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2$$

sujeto a

$$\sum_{j=1}^p |\beta_j| \leq s$$

- Penalización L_1 ¹³.

- Alternativamente,

$$\text{lasso} = \arg \min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \alpha \sum_{j=1}^p |\beta_j| \right\}$$

¹³ La norma L_1 se calcula simplemente como la suma del valor absoluto de los elementos del vector.

Métricas de evaluación

- El **Error Cuadrático Medio (MSE)** mide el promedio de los errores al cuadrado entre los valores observados (y_i) y los valores predichos ($\hat{y}_i$).

Ecuación del MSE

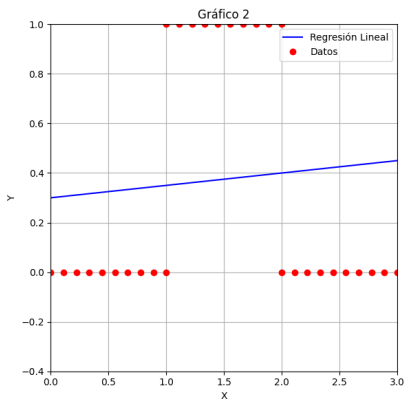
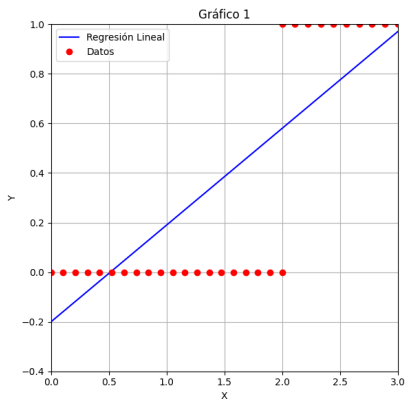
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (5)$$

- El **Coefficiente de Determinación (R^2)** indica la proporción de la varianza en la variable dependiente que es predecible a partir de las variables independientes.
- Un R^2 cercano a 1 indica que el modelo explica bien la variabilidad de los datos.

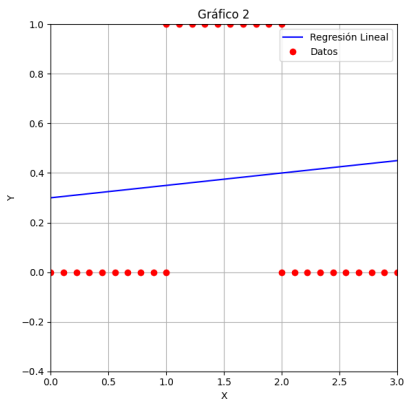
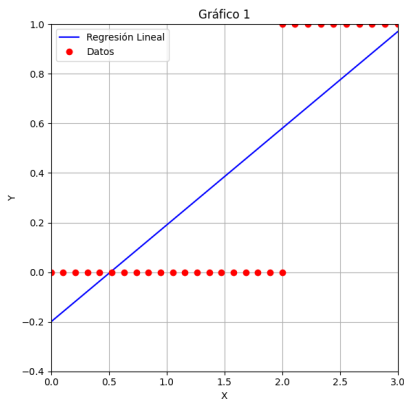
Ecuación del R^2

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (6)$$

¿Por qué no utilizar un algoritmo de regresión lineal?



¿Por qué no utilizar un algoritmo de regresión lineal?



Los modelos de regresión lineal pueden ser inadecuados cuando los datos contienen clases que no son linealmente separables.

Regresión Logística

- ▶ Es un modelo que se utiliza para predecir la probabilidad de cierta clase dado un conjunto de variables independientes.

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

- ▶ **logit**: Transformación logit de la probabilidad p ¹⁴.
- ▶ **Probabilidad de evento de interés**: p representa la probabilidad del evento de interés.
- ▶ **Parámetros**: $\beta_0, \beta_1, \dots, \beta_n$ son los parámetros del modelo.
- ▶ **Variables independientes**: $x_1, x_2, \dots, x_n$.

¹⁴La función logit convierte la probabilidad de un evento (un valor entre 0 y 1) en un valor real que puede variar desde $-\infty$ hasta ∞ .

Demostración de la Función Sigmoide



$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$



$$\frac{p}{1-p} = e^{\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n}$$



$$p = e^{\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n} \cdot (1-p)$$



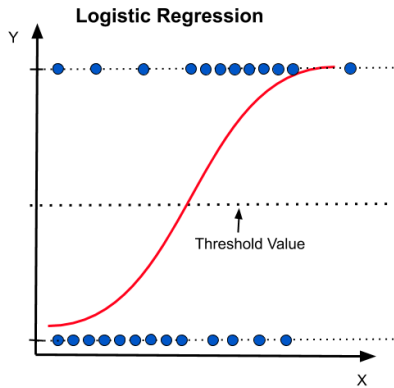
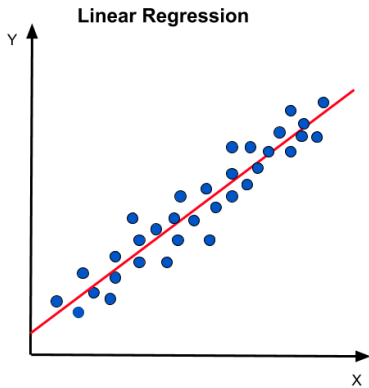
$$p \cdot \left(1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n}\right) = e^{\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n}$$



$$p = \frac{e^{\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n}}$$



$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}} = \frac{1}{1 + e^{-\beta^T x}}$$



- Mide la relación entre la clase y el vector de entrada estimando probabilidades usando una función logística:

$$f_{\beta}(x) = g(\beta^T x) = \frac{1}{1 + e^{-\beta^T x}},$$

- Donde

$$g(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

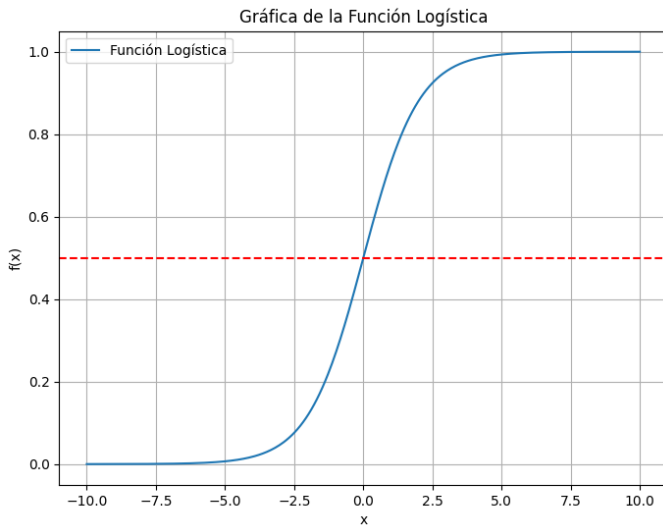
es la función logística o **sigmoide**.

- Como antes,

$$\beta^T x = \beta_0 + \sum_{i=1}^l \beta_i x(i)$$

- Es importante mencionar que $f_{\beta}(x)$ no es lineal.

Función logística



- ▶ Cuando $z = 0 = \beta^T x$, $g(z)$ está en el umbral de decisión 0.5.
- ▶ Cuando $z \rightarrow \infty$, $g(z) \rightarrow 1$.
- ▶ Cuando $z \rightarrow -\infty$, $g(z) \rightarrow 0$.

- La derivada de $g(z)$ es:

$$\begin{aligned} g'(z) &= \frac{d}{dz} \frac{1}{1 + e^{-z}} \\ &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\ &= \frac{1}{(1 + e^{-z})} \cdot \left(1 - \frac{1}{1 + e^{-z}} \right) \\ &= g(z)(1 - g(z)) \end{aligned}$$

- Supongamos que:

$$P(y = 1|x; \beta) = f_{\beta}(x)$$

$$P(y = 0|x; \beta) = 1 - f_{\beta}(x)$$

- Significa que la probabilidad de que y sea 1 dado x y los parámetros β es $f_{\beta}(x)$.

- Por lo tanto:

$$p(y|x; \beta) = (f_{\beta}(x))^y (1 - f_{\beta}(x))^{1-y}$$

- Combina ambas probabilidades en una sola fórmula. Esta expresión se utiliza para calcular la probabilidad conjunta de los datos dados los parámetros del modelo y se maximiza durante la estimación de máxima verosimilitud.

¿Cómo obtener β ?

- Suponiendo que los M ejemplos de entrenamiento son independientes, podemos expresar la función de verosimilitud como¹⁵:

$$\begin{aligned} L(\beta) &= \prod_{m=1}^M p(y_m | x_m; \beta) \\ &= \prod_{m=1}^M (f_{\beta}(x_m))^{y_m} (1 - f_{\beta}(x_m))^{1-y_m} \end{aligned}$$

- Maximizamos la función de log-verosimilitud¹⁶:

$$\begin{aligned} \ell(\beta) &= \sum_{m=1}^M y_m \log(f_{\beta}(x_m)) + (1 - y_m) \log(1 - f_{\beta}(x_m)) \\ &= \sum_{m=1}^M y_m \beta^T x_m - \log(1 + e^{\beta^T x_m}) \end{aligned}$$

¹⁵Se asume que los M ejemplos de entrenamiento son independientes, lo que permite expresar la verosimilitud conjunta como el producto de las verosimilitudes individuales.

¹⁶Para facilitar la optimización, tomamos el logaritmo natural de la función de verosimilitud. Esto convierte el producto en una suma.

Derivación de la Log-Verosimilitud en Regresión Logística

► Paso 1: Definición de la Verosimilitud

$$L(\beta) = \prod_{m=1}^M p(y_m | x_m; \beta) = \prod_{m=1}^M (f_{\beta}(x_m))^{y_m} (1 - f_{\beta}(x_m))^{1-y_m}$$

donde $f_{\beta}(x_m) = \frac{1}{1+e^{-\beta^T x_m}}$ es la función sigmoide.

► Paso 2: Aplicación del Logaritmo Natural

$$\ell(\beta) = \log L(\beta) = \sum_{m=1}^M [y_m \log(f_{\beta}(x_m)) + (1 - y_m) \log(1 - f_{\beta}(x_m))]$$

► Paso 3: Sustitución de la Función Sigmoide

$$f_{\beta}(x_m) = \frac{1}{1 + e^{-\beta^T x_m}} = \frac{e^{\beta^T x_m}}{e^{\beta^T x_m} + 1} \qquad 1 - f_{\beta}(x_m) = \frac{e^{-\beta^T x_m}}{1 + e^{-\beta^T x_m}} = \frac{1}{e^{\beta^T x_m} + 1}$$

► Paso 4: Simplificación de los Logaritmos

$$\log(f_{\beta}(x_m)) = \log(e^{\beta^T x_m}) - \log(e^{\beta^T x_m} + 1)$$

$$\log(1 - f_{\beta}(x_m)) = -\log(e^{\beta^T x_m} + 1)$$

► Resultado Final: Log-Verosimilitud

$$\ell(\beta) = \sum_{m=1}^M [y_m \beta^T x_m - \log(1 + e^{\beta^T x_m})]$$

- Ahora podemos usar el algoritmo de descenso de gradiente:

$$\beta^{p+1} = \beta^p + \alpha \nabla \beta \ell(\beta)$$

- Obtengamos la derivada:

$$\begin{aligned} \frac{\partial}{\partial \beta_i} \ell(\beta) &= \left(\frac{y}{f_\beta(x)} - (1-y) \frac{1}{1-f_\beta(x)} \right) \frac{\partial}{\partial \beta_i} f_\beta(x) \\ &= \left(\frac{y}{f_\beta(x)} - (1-y) \frac{1}{1-f_\beta(x)} \right) f_\beta(x)(1-f_\beta(x)) \frac{\partial}{\partial \beta_i} \beta^T x \\ &= (y(1-f_\beta(x)) - (1-y)f_\beta(x)) x^{(i)} \\ &= (y - f_\beta(x)) x^{(i)} \end{aligned}$$

- Así, la regla de ascenso de gradiente estocástico es:

$$\beta_i^{p+1} = \beta_i^p + \alpha (y - f_\beta(x)) x^{(i)}$$

Función de Log-Verosimilitud:

$$\ell(\beta) = \sum_{m=1}^M \left[y_m \log(f_{\beta}(x_m)) + (1 - y_m) \log(1 - f_{\beta}(x_m)) \right]$$

donde $f_{\beta}(x_m) = \frac{1}{1 + e^{-\beta^T x_m}}.$

Paso 1: Sustitución de $f_{\beta}(x_m)$:

$$\ell(\beta) = \sum_{m=1}^M \left[y_m \beta^T x_m - \log(1 + e^{\beta^T x_m}) \right]$$

Paso 2: Derivada respecto a β :

$$\begin{aligned} \frac{\partial \ell(\beta)}{\partial \beta} &= \sum_{m=1}^M \left[y_m \frac{\partial \beta^T x_m}{\partial \beta} - \frac{\partial}{\partial \beta} \log(1 + e^{\beta^T x_m}) \right] \\ &= \sum_{m=1}^M \left[y_m x_m - \frac{e^{\beta^T x_m}}{1 + e^{\beta^T x_m}} x_m \right] \end{aligned}$$

Regresión Lineal:

- ▶ Predice valores continuos.
- ▶ Relación lineal entre variables.
- ▶ Suceptible al sobreajuste en datos ruidosos.

Regresión Logística:

- ▶ Predice probabilidades de clases.
- ▶ Usa función sigmoide para resultados entre 0 y 1.
- ▶ Ideal para problemas de clasificación binaria.

Perceptrón

Artificial Neural Networks (ANNs)

Los algoritmos de aprendizaje automático tienen como objetivo ajustar un conjunto de parámetros para aprender a partir de un conjunto de datos.

Neurona Artificial (Perceptron)

$$y = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (7)$$

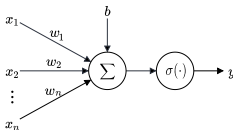


Figura: Representación esquemática de una sola neurona artificial.

Artificial Neural Networks (ANNs)

Los algoritmos de aprendizaje automático tienen como objetivo ajustar un conjunto de parámetros para aprender a partir de un conjunto de datos.

Neurona Artificial (Perceptron)

$$y = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (7)$$

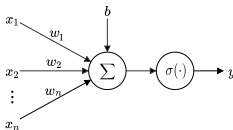


Figura: Representación esquemática de una sola neurona artificial.

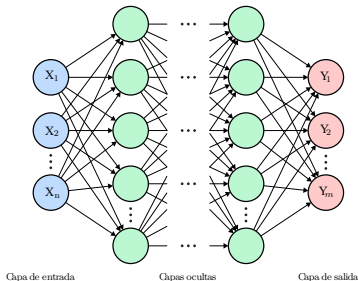
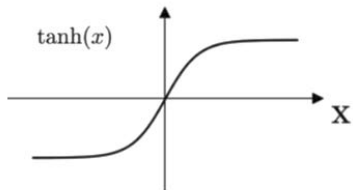
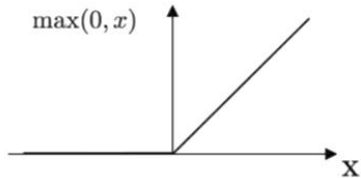


Figura: Estructura generalizada de una red Multi-layer Perceptron (MLP).

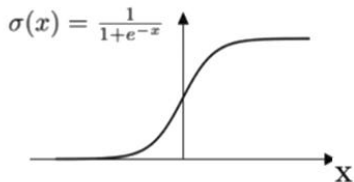
Tanh



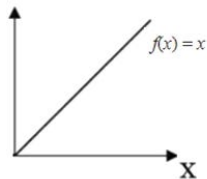
ReLU



Sigmoid



Linear



Validación cruzada

¿Cómo seleccionar un modelo en la práctica?

- ▶ ¿Cómo elegir automáticamente los parámetros del modelo?
- ▶ Asumiremos que tenemos un conjunto finito de modelos $\mathcal{M} = \{M_1, \dots, M_d\}$.
- ▶ En caso de que el(los) parámetro(s) sea(n) continuo(s), podemos discretizarlo(s).
- ▶ Como mencionamos antes, elegir los parámetros que minimizan el error de entrenamiento no garantiza la generalización.

- ▶ **Problema con conjuntos de entrenamiento y prueba fijos:**
 - Un conjunto de prueba pequeño implica una alta incertidumbre estadística.
 - Dificultad para comparar algoritmos debido a la variabilidad en el error estimado.
- ▶ **Datasets extensos**, no presentan este problema.
- ▶ Permiten utilizar todos los ejemplos para estimar el error medio de prueba. Implican un mayor costo computacional.

- ▶ **Problema con conjuntos de entrenamiento y prueba fijos:**
 - Un conjunto de prueba pequeño implica una alta incertidumbre estadística.
 - Dificultad para comparar algoritmos debido a la variabilidad en el error estimado.
- ▶ **Datasets extensos**, no presentan este problema.
- ▶ Permiten utilizar todos los ejemplos para estimar el error medio de prueba. Implican un mayor costo computacional.
- ▶ Uno de los métodos mas conocidos es **k-Fold Cross-Validation**, que divide el conjunto de datos en k subconjuntos no superpuestos. Estimar el error de prueba promedio a partir de los k ensayos.
- ▶ **Consideraciones:**
 - No existen estimadores insesgados para la varianza de estos estimadores de error promedio.
 - Se utilizan aproximaciones para la estimación de la varianza (Bengio y Grandvalet, 2004).

Validación cruzada con conjunto de prueba (Hold-out)

- Una opción es hacer lo siguiente:

Algoritmo: Validación cruzada con conjunto de prueba

1. Dividir aleatoriamente S en S_{train} (por ejemplo, el 75 % de los datos) y S_{cv} (el 25 % restante). Donde S_{cv} es el conjunto de validación cruzada.
2. Entrenar cada modelo M_i en S_{train} solo para obtener la hipótesis f_i .
3. Seleccionar y mostrar la hipótesis f_i con el menor error $\hat{\epsilon}_{S_{\text{cv}}}(f_i)$ en el conjunto de validación cruzada.

Validación cruzada con conjunto de prueba (Hold-out)

- Una opción es hacer lo siguiente:

Algoritmo: Validación cruzada con conjunto de prueba

1. Dividir aleatoriamente S en S_{train} (por ejemplo, el 75 % de los datos) y S_{cv} (el 25 % restante). Donde S_{cv} es el conjunto de validación cruzada.
2. Entrenar cada modelo M_i en S_{train} solo para obtener la hipótesis f_i .
3. Seleccionar y mostrar la hipótesis f_i con el menor error $\hat{\epsilon}_{S_{\text{cv}}}(f_i)$ en el conjunto de validación cruzada.

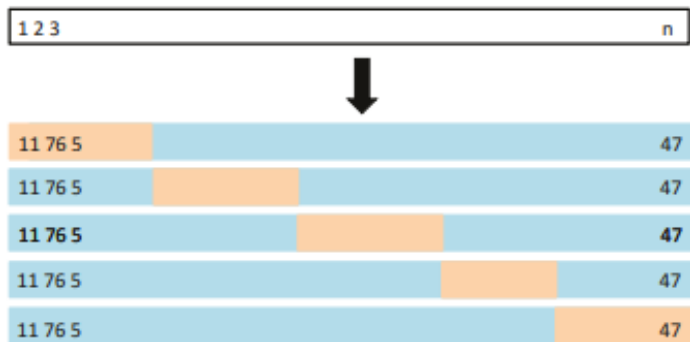
- Así, para cada f_i obtenemos una mejor estimación del error de generalización.
- Y podemos seleccionar la hipótesis con el error de generalización estimado mínimo.
- Usualmente, el tamaño del conjunto de validación cruzada se establece entre 1/4 y 1/3.
- Estamos perdiendo datos que no estamos usando para el entrenamiento. Y estimamos el error de generalización solo a partir de un conjunto de validación cruzada.

Algoritmo: Validación Cruzada K-Fold

1. Dividir aleatoriamente S en K subconjuntos de M/K ejemplos de entrenamiento cada uno. Llamemos a estos subconjuntos $S_1, \dots, S_K$.
2. Para cada modelo M_i hacer:
 - 1) Para $j = 1, \dots, K$ hacer:
 - 1) $f_{ij} \leftarrow A_i(S \setminus S_j)$ (entrenar con todos los datos excepto S_j)
 - 2) $\hat{\epsilon}_{S_j}(f_{ij}) = \frac{1}{|S_j|} \sum_{(x_m, y_m) \in S_j} \ell(y_m, f_{ij}(x_m))$ (calcular el error sobre el conjunto de validación)
 - 2) $\hat{\epsilon}_{M_i} = \frac{1}{K} \sum_{k=1}^K \hat{\epsilon}_{S_k}(f_{ik})$ (calcular el promedio del error de validación sobre los k pliegues)
3. Seleccionar el modelo M_i con el menor $\hat{\epsilon}_{M_i}$.
4. $f \leftarrow A_i(S)$ (entrenar una hipótesis con todos los datos según el modelo M_i).
5. Salida: f

► A_i es el algoritmo de aprendizaje según el modelo M_i .

k-fold cross validation



Validación Cruzada K-Fold (2)

- ▶ Usualmente $K = 5$ o $K = 10$.
- ▶ Cuando $K = M$ este método se llama validación cruzada leave-one-out.
- ▶ En problemas de clasificación, a veces la proporción de ejemplos de cada clase es desigual. En este caso, podemos adaptar la validación cruzada de manera que cada pliegue tenga las mismas proporciones. Así, todos los pliegues estarán igualmente desbalanceados.
- ▶ Este método se llama validación cruzada K-Fold estratificada.

Preguntas